# N057 Expo로 휴대폰 앱 만들기_학습서

Chapter01

## Expo로 휴대폰 앱 만들기

지금 AI 친구 에이전트 서비스는 웹 주소로만 쓸 수 있습니다. 친구처럼 매일 말을 거는 서비스라면 사용자는 브라우저를 열고 주소를 찾기보다 폰 홈 화면의 앱 아이콘을 누르고 싶어 합니다. 그런데 앱을 만들려면 Swift와 Kotlin을 따로 배우고, Android Studio와 Xcode를 설치하고, iOS 앱은 Mac에서만 빌드할 수 있다고 알려져 있습니다. Windows 노트북으로, 코딩 에이전트와 함께, 지금 있는 백엔드를 그대로 쓰는 앱을 만들 수는 없을까요. 오늘은 휴대폰으로 가져가는 방법들을 비교해 고르고, Expo로 대화 앱을 내 폰에서 띄운 뒤 화면을 직접 고치고, 카메라로 찍은 일정표를 읽어 약속으로 만드는 기능까지 붙여 봅니다. 앱 파일을 빌드해 다른 사람 폰에 설치하고 출시를 준비하는 일은 그다음 단계입니다.

**주관식 퀴즈**

### Q. [시작 질문] 웹으로 잘 돌아가는 서비스를 휴대폰 앱으로 만든다고 할 때, 무엇이 새로 필요하고 무엇은 그대로 쓸 수 있을 것 같은지 적어 보세요. 화면, 서버, 데이터베이스, 로그인, 배포로 나눠 생각해 보면 됩니다.

- 
  - /300

  **다음**

## 오늘의 학습 목표

1. 웹 서비스와 앱이 사용자에게 닿고 실행되는 방식의 차이를 말하고, 플랫폼, 네이티브, 빌드가 무엇인지 설명할 수 있습니다.
2. PWA, 웹 래핑, 네이티브, React Native와 Expo의 차이를 설치 부담, iOS 빌드, 기기 기능으로 비교하고 내 서비스에 맞는 방법을 고를 수 있습니다.
3. Expo가 개발, 빌드, 배포를 어떻게 나눠 처리하는지 설명하고, Expo Go와 개발용 빌드의 차이와 한계를 말할 수 있습니다.

4. 앱이 백엔드에서 설정을 받아 로그인하고 답을 스트리밍으로 받는 흐름을 시퀀스 다이어그램으로 읽을 수 있습니다.
5. 템플릿으로 만든 앱을 내 폰의 Expo Go로 실행하고, 연결이 안 될 때 원인을 찾아 해결할 수 있습니다.
6. 바꿀 화면의 구조를 먼저 정한 뒤 코딩 에이전트에게 구현을 맡기고, 저장하면 폰 화면이 바로 바뀌는 것을 확인할 수 있습니다.
7. 사진첩(`expo-image-picker`)과 앱 안 카메라 화면(`expo-camera`)으로 받은 사진을 백엔드 OCR로 읽어 약속으로 만드는 기능을 붙일 수 있습니다.

Chapter02

# 웹 서비스와 앱의 차이

지금까지 만든 에이전트 서비스는 웹 서비스입니다. 사용자가 브라우저에 주소를 넣으면 브라우저가 서버에서 화면 코드(HTML, JavaScript)를 받아 그 자리에서 실행하고, 탭을 닫으면 그 코드는 사라집니다. 다음에 주소를 열면 서버에서 다시 받으므로, 서버에 새 버전을 배포하면 모든 사용자가 곧바로 새 화면을 봅니다.

앱(app)은 휴대폰에 설치해 두고 실행하는 프로그램입니다. 앱 파일 안에는 화면을 그리고 동작하는 코드, 아이콘과 이미지, 앱 이름과 권한 같은 설정이 함께 들어 있고, 사용자는 스토어에서 이 파일을 한 번 내려받아 설치합니다. 홈 화면의 아이콘을 누르면 폰이 설치된 파일을 실행하므로 화면 코드를 서버에서 받아 올 필요가 없고, 대화나 크레딧처럼 바뀌는 데이터만 백엔드 API로 받습니다. 대신 화면을 고치면 앱 파일을 새로 만들어 스토어에 올리고 사용자가 업데이트해야 바뀌며, 업데이트하지 않은 사용자의 폰에는 옛 버전이 계속 남아 있습니다.

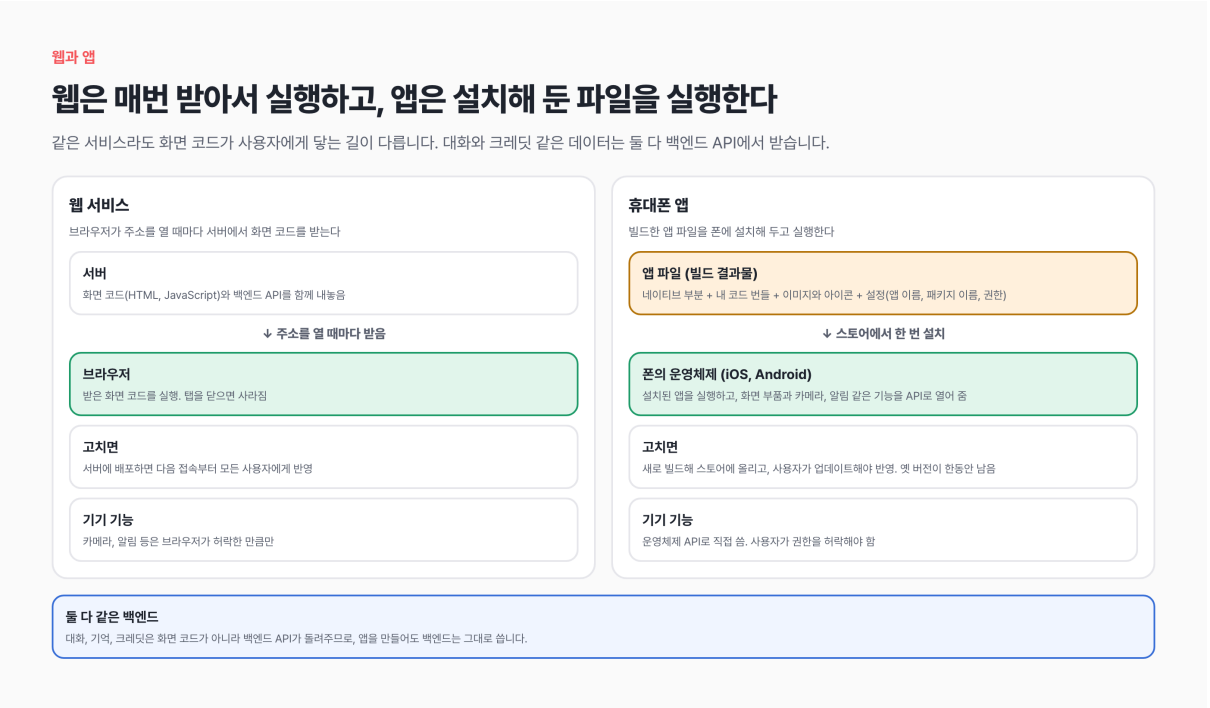



*그림: 웹 서비스는 주소를 열 때마다 서버에서 화면 코드를 받고, 앱은 빌드한 앱 파일을 설치해 두고 실행한다. 데이터는 둘 다 백엔드 API에서 받는다*

그림의 아래 칸처럼 화면이 웹에서 앱으로 바뀌어도 대화, 기억, 크레딧은 백엔드가 돌려주므로 백엔드는 그대로 씁니다. 달라지는 점은 옛 버전 앱이 사용자 폰에 남는다는 것입니다. 웹은 화면과 서버를 함께 배포하면 끝나지만, 앱을 내놓은 뒤에는 옛 버전 화면이 새 백엔드를 부르는 상황이 생깁니다. 그래서 앱이 쓰는 백엔드 API의 응답 모양은 옛 앱이 읽지 못하게 함부로 바꾸지 않습니다.

# 운영체제와 플랫폼

휴대폰 앱은 폰의 운영체제(OS, operating system) 위에서 실행됩니다. 운영체제는 화면, 터치, 네트워크, 카메라 같은 폰의 자원을 관리하고, 앱이 그 자원을 쓸 수 있도록 API를 열어 주는 기본 프로그램입니다. 휴대폰 운영체제는 Apple의 iOS와 Google의 Android가 대부분을 차지합니다. 앱 개발에서는 운영체제와 그 위에서 앱을 만드는 도구, 앱을 내보내는 스토어를 묶어 플랫폼이라고 부릅니다.

| | iOS | Android |
|---|---|---|
| 기기 | iPhone, iPad. Apple만 만듦 | 삼성, Google 등 여러 제조사가 만듦 |
| 공식 언어 | Swift | Kotlin |
| 개발 도구 | Xcode. Mac에서만 실행됨 | Android Studio. Windows, Mac, Linux에서 실행됨 |

| | iOS | Android |
|---|---|---|
| 앱 파일 | IPA | APK(직접 설치용), AAB(스토어 제출용) |
| 스토어 | App Store | Google Play |
| 스토어 밖 설치 | 기본적으로 막혀 있음 | 사용자가 허용하면 APK를 직접 설치할 수 있음 |

두 플랫폼은 언어도 도구도 달라서, 원래는 같은 앱을 플랫폼마다 한 번씩 만들어야 합니다. 플랫폼마다 앱을 만드는 데 쓰는 라이브러리와 도구 묶음을 SDK(software development kit)라고 하고, 운영체제가 새 버전을 낼 때마다 SDK도 함께 바뀝니다.

# 네이티브라는 말

네이티브(native)는 플랫폼이 기본으로 제공하는 언어와 부품으로 만든 것을 가리킵니다. iPhone 설정 앱의 켜고 끄는 스위치, 목록을 밀어 올릴 때의 스크롤, 글자를 입력할 때 올라오는 키보드는 iOS가 제공하는 네이티브 화면 요소입니다. 카메라, 위치, 알림, 연락처처럼 운영체제가 API로 열어 주는 기능은 네이티브 기능이라고 부르는데, 네이티브 기능은 사용자가 권한을 허락해야 쓸 수 있습니다. 앱이 처음 카메라를 쓸 때 "카메라 접근을 허용하시겠습니까" 창이 뜨는 이유입니다.

네이티브와 견주는 것이 웹 화면입니다. 앱 안에 브라우저 창을 하나 띄워 웹 페이지를 보여 주는 부품을 웹뷰(WebView)라고 하는데, 웹뷰로 그린 화면은 생김새와 반응이 브라우저와 같습니다. 그래서 "네이티브 앱"이라고 하면 보통 화면을 네이티브 요소로 그리는 앱을 뜻합니다. 오늘 쓰는 React Native는 코드는 JavaScript로 쓰지만 화면은 각 플랫폼의 네이티브 요소로 그리므로, 한 코드로 iOS와 Android의 네이티브 화면을 함께 만듭니다. 섹션 3에서는 화면을 무엇으로 그리는지를 기준으로 휴대폰으로 가져가는 방법 네 가지를 비교합니다.

# 빌드와 앱 파일

빌드(build)는 사람이 쓴 소스 코드를 폰이 설치하고 실행할 수 있는 앱 파일로 만드는 과정입니다. 빌드하는 컴퓨터는 아래 순서로 앱 파일을 만듭니다.

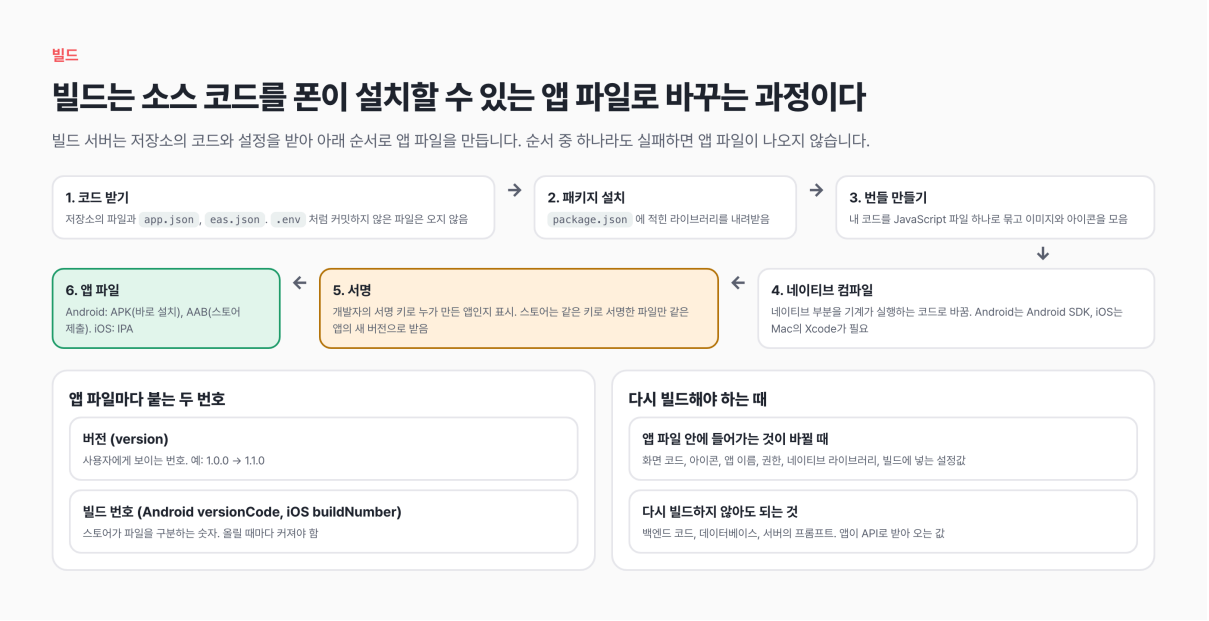



*그림: 빌드는 코드를 받고, 패키지를 설치하고, 번들을 만들고, 네이티브 부분을 컴파일하고, 서명해 앱 파일을 만든다*

그림의 3번 번들(bundle)은 내 JavaScript 코드를 실행에 필요한 파일 하나로 묶은 것이고, 4번 컴파일(compile)은 네이티브 부분의 코드를 기계가 실행하는 형태로 바꾸는 일입니다. 5번 서명은 개발자의 키로 누가 만든 파일인지 표시하는 일이라, 폰과 스토어는 서명을 보고 파일이 중간에 바뀌지 않았는지, 같은 개발자가 낸 새 버전인지 확인합니다. 아래쪽 칸처럼 앱 파일 안에 들어가는 것을 바꾸면 다시 빌드해야 하고, 백엔드처럼 바깥에 있는 것은 고쳐도 다시 빌드하지 않습니다. 앱 빌드의 결과물은 사용자 수만큼의 폰에 설치되므로, 한 번 내보낸 앱 파일은 되돌릴 수 없습니다.

# 빌드에 Android SDK와 Xcode가 필요한 이유

그림의 4번과 5번은 플랫폼 회사가 내놓은 도구로만 할 수 있습니다. 네이티브 부분의 코드(Kotlin, Swift 등)를 폰의 CPU가 실행하는 기계어로 바꾸는 컴파일러, 버튼이나 카메라처럼 운영체제가 제공하는 기능의 정의가 담긴 플랫폼 SDK, APK와 IPA라는 정해진 형식으로 묶는 도구, 서명 도구가 모두 이 안에 들어 있기 때문입니다. Android는 실제 빌드를 Android SDK와 Gradle이 하고 Android Studio는 그 도구를 쓰기 편하게 해 주는 편집기라서, SDK만 설치하면 Windows, Mac, Linux 어디서나 빌드할 수 있습니다. iOS는 SDK, 컴파일러, 서명 도구가 모두 Xcode에 묶여 있고 Xcode는 macOS에서만 돌며, iOS 서명에는 Apple이 발급한 인증서까지 필요해서 결국 어딘가에 Mac이 있어야 합니다.

그래서 원래는 Android 앱을 만들려면 Android SDK를, iOS 앱을 만들려면 Mac과 Xcode를 내 컴퓨터에 갖춰야 합니다. 오늘 쓰는 Expo도 이 단계를 건너뛰지는 않고, 미리 컴파일해 둔 앱과 클라우드 빌드 서버로 그 일을 내 노트북 밖으로 옮깁니다. 어떻게 나누는지는 섹션 4에서 봅니다.

# 앱이 실행되는 동안

앱은 켜져 있는 동안에도 상태가 바뀝니다. 사용자가 홈 화면으로 나가거나 다른 앱을 열면 앱은 백그라운드로 내려가고, 운영체제는 메모리가 부족하면 백그라운드의 앱을 예고 없이 종료합니다. 그래서 답을 받는 도중에 앱을 내리면 스트리밍이 끊길 수 있고, 다시 열었을 때 화면을 처음부터 그려야 할 수 있습니다. 앱을 꺼도 남아야 하는 로그인 정보 같은 값은 폰의 저장소에 두고, 대화 기록처럼 다른 기기에서도 보여야 하는 값은 백엔드에 둡니다. 섹션 6에서 템플릿이 어떤 값을 폰에 두고 어떤 값을 백엔드에서 받는지 확인합니다.

| 자주 듣는 말 | 뜻 |
| --- | --- |
| 플랫폼 | 운영체제와 개발 도구, 스토어를 묶어 부르는 말. iOS와 Android |
| SDK | 플랫폼이나 프레임워크가 제공하는 개발용 라이브러리와 도구 묶음 |
| 네이티브 | 플랫폼이 기본으로 제공하는 언어, 화면 요소, 기능 |
| 웹뷰 | 앱 안에서 웹 페이지를 보여 주는 브라우저 창 부품 |
| 패키지 이름 | 스토어와 폰이 앱을 구분하는 고유 이름. 예: `kr.modu.harufriend` |
| 권한 | 카메라, 위치, 알림처럼 사용자가 허락해야 쓰는 기능 |

**객관식 퀴즈**

### Q. 백엔드 API 응답의 필드 이름을 새 이름으로 바꾸고, 앱도 새 이름을 읽도록 고쳐 스토어에 새 버전을 올리려고 합니다. 웹 화면만 있을 때는 없던 문제로 가장 알맞은 것은 무엇인가요?

• 1. 백엔드를 바꿀 때마다 앱도 스토어 심사를 다시 받아야 한다
• 2. 업데이트하지 않은 사용자의 폰에서는 옛 앱이 옛 필드 이름을 찾으므로, 그 사용자의 화면이 깨진다
• 3. 앱은 백엔드 API를 부르지 않고 앱 파일 안의 데이터를 쓰므로 필드 이름이 상관없다
• 4. 필드 이름을 바꾸면 앱의 서명이 달라져 설치가 막힌다

- •

  1. 백엔드를 바꿀 때마다 앱도 스토어 심사를 다시 받아야 한다

- •

  - . 업데이트하지 않은 사용자의 폰에서는 옛 앱이 옛 필드 이름을 찾으므로, 그 사용자의 화면이 깨진다

- •

  3. 앱은 백엔드 API를 부르지 않고 앱 파일 안의 데이터를 쓰므로 필드 이름이 상관없다

- •

4. 필드 이름을 바꾸면 앱의 서명이 달라져 설치가 막힌다

Chapter03

# 네 가지 방법

웹 서비스를 휴대폰으로 가져가는 방법은 크게 네 가지입니다. 무엇으로 화면을 그리는지, 그리고 스토어에 올릴 앱 파일을 어디서 만드는지가 다릅니다.

PWA(Progressive Web App)는 웹 페이지에 설정 파일과 서비스 워커를 더해, 브라우저에서 "홈 화면에 추가"로 앱처럼 설치하게 하는 방식입니다. 따로 빌드하지 않으므로 가장 가볍지만, 스토어에 올라가지 않고 알림 같은 기기 기능은 브라우저가 허용하는 만큼만 씁니다. 웹 래핑(Capacitor 등)은 웹 페이지를 앱 껍데기에 넣어 스토어용 앱 파일로 만드는 방식입니다. 화면은 여전히 웹이고, 앱 파일을 만들려면 내 컴퓨터에 Android Studio와 Xcode가 있어야 합니다.

네이티브는 iOS는 Swift, Android는 Kotlin으로 각각 만들거나, Flutter처럼 한 코드로 두 플랫폼의 네이티브 앱을 만드는 방식입니다. 폰의 모든 기능을 쓸 수 있지만, 빌드 도구를 내 컴퓨터에 설치해야 하고 iOS 앱은 Mac에서만 빌드할 수 있습니다. React Native는 JavaScript로 코드를 쓰면 폰의 네이티브 화면 요소로 그려 주는 프레임워크이고, Expo는 React Native 앱을 만들고 빌드하고 배포하는 과정을 묶어 주는 도구 모음입니다. Expo는 앱 파일을 클라우드에서 빌드하므로, 내 컴퓨터에는 Node.js만 있으면 되고 Windows에서도 iOS 앱 파일을 만들 수 있습니다.

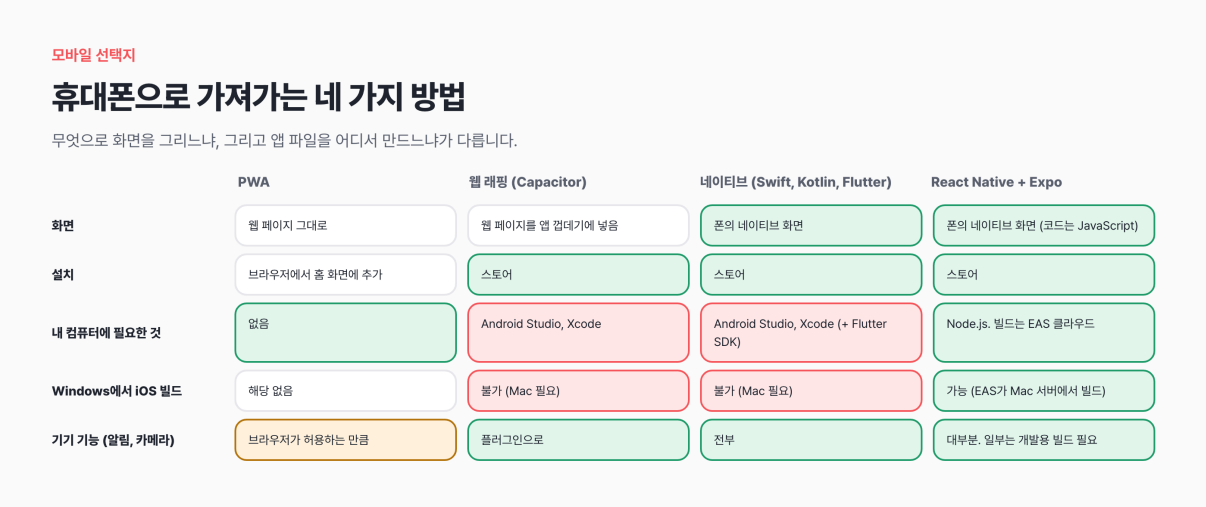



*그림: PWA, 웹 래핑, 네이티브, React Native와 Expo를 화면, 설치, 필요한 도구, iOS 빌드, 기기 기능으로 비교한 표*

# 고르는 기준

어느 방법이 좋은지는 상황마다 다릅니다. 스토어에 올리지 않아도 되고 웹 화면을 그대로 쓰고 싶다면 PWA가 가장 빠릅니다. 이미 잘 만든 웹 화면이 있고 Mac과 빌드 도구를 갖춘 팀이라면 웹 래핑도 해 볼 만합니다. 게임처럼 그래픽이 무겁거나 최신 OS 기능을 바로 써야 한다면 네이티브가 맞습니다.

이 과정의 조건은 세 가지입니다. 수강생 노트북은 대부분 Windows이고, 코딩 에이전트와 함께 만들며, 백엔드는 이미 있어서 앱은 화면만 새로 만들면 됩니다. 이 조건에서 iOS까지 다룰 수 있으면서 내 컴퓨터에 설치할 것이 가장 적은 방법이 Expo라서, 오늘은 Expo로 진행합니다. 다만 Expo도 앱 파일은 결국 컴파일해야 하므로, 그 일을 클라우드 서비스(EAS)에 맡긴다는 점과 무료 플랜의 빌드 횟수 제한은 감수합니다.

**객관식 퀴즈**

### Q. PWA와 웹 래핑(Capacitor 등)은 둘 다 화면을 웹으로 그립니다. 두 방법의 차이로 맞는 것은 무엇인가요?

• 1. PWA는 스토어에 올라가고, 웹 래핑은 브라우저의 홈 화면에 추가로만 설치한다
• 2. 웹 래핑은 스토어에 올릴 앱 파일을 만들기 때문에, 내 컴퓨터에 Android Studio와 Xcode가 필요하다
• 3. 웹 래핑은 웹 화면을 네이티브 화면 요소로 바꿔 그린다
• 4. PWA는 카메라와 알림을 포함해 폰의 모든 기능을 쓸 수 있다

- •

  1. PWA는 스토어에 올라가고, 웹 래핑은 브라우저의 홈 화면에 추가로만 설치한다

- •
  - . 웹 래핑은 스토어에 올릴 앱 파일을 만들기 때문에, 내 컴퓨터에 Android Studio와 Xcode가 필요하다

- •

  3. 웹 래핑은 웹 화면을 네이티브 화면 요소로 바꿔 그린다

- •

  4. PWA는 카메라와 알림을 포함해 폰의 모든 기능을 쓸 수 있다

Chapter04

# Expo 정의

Expo는 React Native로 Android와 iOS 앱을 만들 때 필요한 개발 도구, 기기 기능 라이브러리, 클라우드 서비스를 한데 묶은 오픈 소스 프레임워크입니다. React Native는 JavaScript 코드로 네이티브 화면을 그리는 일까지만 맡기 때문에, React Native만으로 앱을 만들면 화면 이동, 카메라 같은 기기 기능, 네이티브 프로젝트 설정, 빌드와 서명에 쓸 도구를 하나씩 골라 직접 맞춰야 합니다. Expo는 이 도구들을 서로 버전이 맞는 한 묶음으로 제공합니다. React Native 공식 문서도 새 앱은 Expo 같은 프레임워크로 시작하기를 권합니다(2026년 11월 기준).

Expo가 제공하는 것을 정리하면 다음과 같습니다.

| 구성 요소 | 하는 일 |
| --- | --- |
| Expo CLI와 개발 서버 | `npx expo start` 로 개발 서버를 띄웁니다. 개발 서버 안의 Metro 번들러가 TypeScript 코드를 JavaScript 번들로 묶어 폰에 보내고, 파일을 저장하면 바뀐 부분만 다시 보냅니다. 터미널에 폰으로 접속할 QR 코드를 띄웁니다 |
| Expo SDK | 카메라, 사진첩, 파일, 알림 같은 기기 기능을 Android와 iOS에서 같은 코드로 쓰게 해 주는 라이브러리 묶음입니다. 예: `expo-camera`, `expo-image-picker` |
| Expo Router | `app` 폴더의 파일 하나를 화면 하나로 만드는 화면 이동 라이브러리입니다 |
| 앱 설정(app config) | 앱 이름, 아이콘, 패키지 이름, 권한 설명을 `app.json` 한 파일에 적습니다 |
| Expo Go | 개발 서버의 번들을 받아 실행하는 개발용 앱입니다. 스토어에서 폰에 설치합니다 |
| Expo Services | 클라우드 빌드(Expo Build), 스토어 제출(Expo Submit), JavaScript만 바꿔 보내는 업데이트(Expo Update)를 맡는 클라우드 서비스입니다. 무료 플랜과 유료 플랜이 있고, 빌드 설정은 `eas.json` 에 적습니다 |

Expo Services는 EAS(Expo Application Services)라는 이름으로 오래 쓰였고, 2026년 11월 기준 공식 문서는 Expo Build, Expo Update처럼 Expo를 붙인 이름으로 바꿔 부릅니다. 명령어(`eas build`)와 설정 파일(`eas.json`)은 이름이 그대로라서, 이 교안에서는 명령어와 맞춰 EAS Build라고 부릅니다.

# Expo로 줄어드는 작업

React Native만으로 시작하면 첫 화면을 보기 전에 Android Studio와 Xcode를 설치하고 네이티브 프로젝트를 한 번 빌드해야 합니다. Expo로 시작하면 노트북에는 Node.js, 폰에는 Expo Go만 있으면 되고, 개발 서버의 QR 코드를 찍는 것으로 첫 화면이 뜹니다. iOS 앱도 Mac 없이 이 방식으로 확인합니다.

네이티브 프로젝트를 관리하는 방식도 다릅니다. 일반 React Native 프로젝트는 `android/` 와 `ios/` 폴더를 저장소에 두고, 카메라 권한 하나를 더할 때도 Android의

`AndroidManifest.xml` 과 iOS의 `Info.plist` 를 따로 고칩니다. Expo 프로젝트는 권한 설명을 `app.json` 에 한 번 적고, 빌드할 때 Expo가 `app.json` 을 읽어 두 폴더를 새로 만듭니다. Expo는 이 방식을 CNG(Continuous Native Generation)라고 부르고, 폴더를 만드는 명령이 `npx expo prebuild` 입니다. 두 폴더는 언제든 다시 만들 수 있으므로 보통 저장소에 올리지 않고 직접 고치지도 않습니다. 코딩 에이전트가 고칠 곳이 `app.json` 한 파일로 모이므로, 승인할 때 무엇이 바뀌는지 읽기도 쉬워집니다.

라이브러리 버전을 맞추는 일도 줄어듭니다. Expo SDK 버전 하나가 React Native 버전과 Expo 라이브러리들의 버전 조합을 정해 두고, `npx expo install` 은 프로젝트의 SDK에 맞는 버전을 골라 설치합니다. 빌드할 때는 EAS Build가 서명 키를 만들어 Expo 서버에 보관하므로, 키 파일을 노트북이나 저장소에 둘 필요가 없습니다.

# Expo를 쓰는 대가

이 편의에는 조건이 붙습니다. Expo Go에 들어 있지 않은 네이티브 기능을 쓰려면 개발용 빌드를 따로 만들어야 하고(아래 "Expo Go와 개발용 빌드"), 클라우드 빌드는 무료 플랜에서 한 달에 쓸 수 있는 횟수가 정해져 있습니다. 빌드 환경과 서명 키를 Expo라는 회사의 서비스에 맡긴다는 점도 고려해야 합니다. Expo Update로 스토어 심사 없이 바꿀 수 있는 것은 JavaScript 코드와 이미지 같은 네이티브가 아닌 부분뿐이라서, 네이티브 라이브러리를 더하거나 `app.json` 의 권한을 바꾸면 앱 파일을 다시 빌드해 스토어에 올려야 합니다.

# 컴파일이 필요한 부분과 필요 없는 부분

섹션 2에서 본 것처럼 React Native 앱은 화면을 그리고 기기 기능을 쓰는 네이티브 부분과 내 코드(JavaScript, TypeScript)로 되어 있고, 플랫폼 도구로 컴파일해야 하는 것은 네이티브 부분뿐입니다. 내 코드는 번들로 묶기만 하면 됩니다.

Expo는 컴파일이 필요한 부분을 두 가지 방법으로 피합니다. 개발할 때는 Expo Go를 씁니다. Expo Go는 자주 쓰는 네이티브 부분을 미리 컴파일해 둔 앱이라서, 내 노트북의 개발 서버(`npx expo start`)가 만든 번들을 와이파이로 받아 바로 실행합니다. 코드를 고치고 저장하면 폰 화면이 몇 초 안에 바뀝니다. 다른 사람에게 줄 앱 파일을 만들 때는 EAS Build를 씁니다. 프로젝트를 Expo의 클라우드로 올리면, Android SDK가 깔린 서버와 Xcode가 깔린 Mac 서버가 컴파일하고 서명해 설치 링크를 돌려줍니다. 그래서 내 컴퓨터에는 Node.js만 있으면 됩니다.




*그림: 개발할 때는 Expo Go가 번들을 받아 실행하고, 빌드할 때는 EAS 클라우드가 컴파일하고, 배포할 때는 스토어를 거친다*

# Expo Go와 개발용 빌드

Expo Go에는 미리 컴파일해 둔 네이티브 기능만 들어 있습니다. 그 밖의 네이티브 라이브러리를 쓰려면 내 앱 전용으로 네이티브 부분까지 컴파일한 개발용 빌드(development build)를 EAS로 만들어 폰에 설치해야 합니다. 개발용 빌드를 설치한 뒤에는 Expo Go와 똑같이 개발 서버에 연결해 저장하면 바로 바뀌는 흐름으로 일하고, 네이티브 라이브러리를 추가하거나 `app.json` 의 권한을 바꿨을 때만 다시 빌드합니다.

| 쓰려는 기능 | Expo Go | 개발용 빌드 |
|---|---|---|
| 화면, 서버 호출, 로그인, 스트리밍 | 됨 | 됨 |
| 카메라 촬영, 사진첩에서 고르기 | 됨 | 됨 |
| 기기 안에서 예약하는 알림(로컬 알림) | 됨 | 됨 |
| 서버가 보내는 푸시 알림(Android) | 안 됨 (SDK 53부터) | 됨. Firebase 설정이 따로 필요 |
| 앱 안 결제, 카카오 로그인 같은 회사별 SDK | 안 됨 | 됨 |
| 홈 화면 위젯, 애플워치 앱 | 안 됨 | 네이티브 코드를 따로 써야 함 |

오늘 만드는 대화 앱은 서버를 부르고 화면을 보여 주며 카메라와 사진첩을 쓰는 앱이라 Expo Go로 충분합니다. 코딩 에이전트가 새 라이브러리를 설치하자고 하면, 그 라이브러리가 Expo Go에서 도는지 먼저 확인하게 합니다. 템플릿의 작업 규칙( `AGENTS.md` )에 "패키지는 `npx expo install` 로 설치하고, Expo 문서를 근거로 확인한다"고 적혀 있는 이유입니다.

# SDK 버전 맞추기

Expo는 1년에 몇 번 새 SDK를 내고, 그때마다 바뀌는 API가 있습니다. 프로젝트가 쓰는 SDK 버전은 `package.json` 의 `expo` 버전으로 정해지고, 폰의 Expo Go가 그 SDK를 지원해야 앱이 열립니다. 스토어의 Expo Go는 보통 최신 SDK 하나만 지원하므로, 프로젝트와 Expo Go의 SDK 버전이 같아야 실행됩니다. 오늘 실습할 내용은 SDK 58에 맞춰 준비했고, 2026년 10월 기준으로 Play 스토어와 App Store의 Expo Go 모두 SDK 58을 지원합니다.

Expo가 새 SDK를 내면 스토어의 Expo Go가 먼저 새 버전으로 올라가서, 이전 SDK로 만든 프로젝트가 열리지 않을 수 있습니다. 반대로 App Store는 Apple 심사 때문에 Expo Go가 늦게 올라오거나 한동안 멈춰 있던 적도 있습니다. 그래서 실행하기 전에 Expo Go 설정 화면의 SDK 버전이 프로젝트의 SDK 버전과 같은지 먼저 봅니다. 버전이 다르면 "버전이 맞지 않는다"는 오류가 뜹니다.

**객관식 퀴즈**

### Q. 오늘 만드는 앱에 아래 기능을 더하려고 합니다. Expo Go로는 확인할 수 없고 개발용 빌드가 있어야 하는 것은 무엇인가요?

- 1. 앱 안 카메라 화면으로 일정표 찍기
- 2. 내일 아침 8시에 울리도록 기기 안에서 알림 예약하기
- 3. 카카오 로그인 SDK로 로그인하기
- 4. 답을 글자 단위로 스트리밍해 보여 주기**제출**

- •

  1. 앱 안 카메라 화면으로 일정표 찍기

- •

  - . 내일 아침 8시에 울리도록 기기 안에서 알림 예약하기

- •

  3. 카카오 로그인 SDK로 로그인하기

- •

  4. 답을 글자 단위로 스트리밍해 보여 주기

## 📚 Expo 참고 자료

- Expo: Expo Go and development builds: Expo Go와 개발용 빌드가 어떻게 다르고 언제 넘어가야 하는지 설명합니다(영어).

- Expo: Set up your environment: 기기와 운영체제별로 Expo Go를 설치하고 프로젝트를 여는 방법이 있습니다(영어).

Chapter05

# 시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 앱은 화면만 새로 만들고 백엔드는 지금 있는 것을 그대로 쓰므로, 백엔드는 공개 주소로 열리기만 하면 됩니다. 내 백엔드가 없거나 망가졌다면 강사가 알려 주는 시연용 백엔드 주소로 시작해도 오늘 실습은 모두 할 수 있습니다.

| 필요한 상태 | 확인할 것 | 없으면 |
|---|---|---|
| Node.js 22.13 이상 (LTS 24 권장) | 터미널에서 `node -v` | 아래 "Node.js 설치" |
| 폰과 Expo Go | 폰에 Expo Go 앱이 있고 설정 화면의 SDK가 58 | Android는 Play 스토어, iPhone은 App Store에서 설치 |
| 공개 주소로 열리는 백엔드 | 브라우저로 `<백엔드 주소>/health` 를 열면 `{"status":"ok"}` | 강사의 시연용 백엔드 주소 |
| (백엔드가 Supabase 로그인을 쓴다면) 익명 로그인 켜짐 | 백엔드 웹 화면 왼쪽 아래에 "게스트로 사용 중" | Supabase > Authentication > Sign In / Providers 에서 Allow anonymous sign-ins 켜고 저장 |
| GitHub 계정, Expo 계정 | github.com, expo.dev 에 로그인됨 | 각 사이트에서 가입. Expo 계정은 무료 |

# Node.js 설치

Expo SDK 58은 Node.js 22.13 이상을 요구합니다. `node -v` 가 이보다 낮거나 명령을 찾지 못하면 설치합니다. 설치 프로그램은 공식 사이트(nodejs.org)의 LTS 버전을 받으면 되고, Windows에서는 PowerShell 명령 한 줄로도 설치할 수 있습니다. 설치한 뒤에는 터미널을 새로 열어야 `node` 명령이 잡힙니다.

**직접 실행할 명령: Node.js 설치와 확인**

```
# Windows PowerShell (winget 이 있는 Windows 10, 11)
winget install OpenJS.NodeJS.LTS

# macOS (Homebrew 를 쓴다면)
```

```
brew install node

# 새 터미널에서 확인
node -v
npm -v
```

Windows PowerShell에서 `npx` 나 `npm` 을 실행했을 때 "이 시스템에서 스크립트를 실행할 수 없으므로 npx.ps1 파일을 로드할 수 없습니다"가 나오면, PowerShell이 내려받은 스크립트 실행을 막고 있는 것입니다. `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` 를 한 번 실행하면 내 계정에서만 로컬 스크립트 실행을 허용합니다. 정책을 바꾸기 싫다면 명령 이름을 `npx.cmd`, `npm.cmd` 로 바꿔 실행해도 됩니다.

# 템플릿으로 내 앱 저장소 만들기

앱 템플릿은 백엔드와 별도 저장소입니다(friend-agent-mobile-template). 백엔드는 Python, 앱은 Node.js로 쓰는 언어와 도구가 달라서 저장소를 나눴습니다. GitHub에서 템플릿 저장소의 Use this template > Create a new repository 로 내 계정에 공개 저장소를 만들고, 내 컴퓨터로 내려받아 패키지를 설치합니다.

**직접 실행할 명령: 템플릿으로 저장소 만들고 내려받기**

```
# 내 계정에 friend-agent-mobile 저장소를 만들고 바로 내려받는다
gh repo create friend-agent-mobile --template 88chacha/friend-agent-mobile-template --
public --clone
cd friend-agent-mobile

# 패키지 설치 (처음 한 번, 몇 분 걸림)
npm install
```

**코딩 에이전트에게 맡길 때: 템플릿으로 저장소 만들기**

```
88chacha/friend-agent-mobile-template 템플릿으로 내 GitHub 계정에 friend-agent-mobile 공개
저장소를 만들고 내려받아 줘.
그다음 그 폴더에서 npm install 을 실행해 줘. 다른 파일은 바꾸지 마.
```

실행 전에 저장소 이름과 공개 여부를 봅니다. 내려받은 폴더의 `README.md` 와 `AGENTS.md` 를 한 번 훑어봅니다. `AGENTS.md` 는 이 저장소에서 코딩 에이전트가 따르는 작업 규칙이고, 위쪽은 이 과정에서 정한 규칙(구조, 비밀값, 빌드 확인), 아래쪽은 Expo가 템플릿에 기본으로 넣는 안내(SDK가 바뀌면 문서를 다시 보라는 내용)입니다.

# Expo 스킬 설치하기

스킬(skill)은 코딩 에이전트가 특정 작업을 할 때 읽는 작업 안내서입니다. 에이전트는 요청과 관련된 스킬이 있으면 그 안내를 먼저 읽고 그대로 작업합니다. Expo 팀은 Expo Router 화면 구성, `npx expo install` 로 SDK에 맞는 패키지 설치하기, SDK 업그레이드, 빌드와 스토어 제출 같은 작업을 Expo가 권하는 방식으로 하게 하는 공식 스킬을 공개해 두었습니다. 에이전트가 학습한 시점의 옛 Expo 사용법 대신 지금 SDK에 맞는 방식으로 코드를 쓰게 하려고 스킬을 설치합니다.

Claude Code에서는 Expo 스킬이 공식 플러그인 마켓플레이스의 `expo` 플러그인으로 묶여 있습니다. 플러그인은 앱 폴더에 프로젝트 범위로 설치합니다. 프로젝트 범위로 설치하면 설정이 저장소의 `.claude/settings.json` 에 기록되어, 같은 저장소를 내려받은 사람의 에이전트도 같은 스킬을 씁니다. 템플릿의 `.claude/settings.json` 에는 이 플러그인을 켜는 설정이 이미 들어 있어서, 설치가 끝나도 이 파일에 새로 바뀌는 내용은 없습니다.

**직접 실행할 명령: Expo 스킬 설치 (앱 폴더에서)**

```
# Claude Code: 앱 폴더에 프로젝트 범위로 플러그인을 설치한다
claude plugin install expo@claude-plugins-official --scope project

# Claude Code 안에서는 이렇게 실행하고, 설치 범위는 프로젝트를 고른다
/plugin install expo@claude-plugins-official

# Codex: 공식 마켓플레이스의 플러그인으로 설치한다
codex plugin add expo@openai-curated

# 그 밖의 코딩 에이전트(Cursor, VS Code의 GitHub Copilot 등): skills CLI 로 설치한다
npx skills@latest add expo/skills --skill '*'
```

skills CLI는 어느 에이전트에 설치할지 묻고, 앱 폴더 안에 스킬 파일(`SKILL.md`)을 넣어 둡니다. 설치한 뒤 에이전트를 다시 시작하면 스킬을 읽기 시작합니다. 설치가 끝나면 에이전트에게 "설치된 Expo 스킬 목록을 보여 줘"라고 물어 스킬이 보이는지 확인합니다. Expo는 빌드 상태와 로그를 에이전트가 직접 조회하게 하는 MCP 서버(`https://mcp.expo.dev/mcp`)도 제공하지만, 오늘은 빌드를 하지 않으므로 스킬만 설치합니다.

Chapter06

# 템플릿의 구조

앱은 화면만 가지고, 데이터와 판단은 모두 백엔드에 있습니다. 웹 화면이 하던 일을 폰 화면이 대신 하는 것이라, 백엔드 API는 하나도 바꾸지 않았습니다. 템플릿의 파일은 아래처럼 역할이 나뉩니다.

| 파일 | 하는 일 |
| --- | --- |
| `src/app/_layout.tsx` | 앱 전체 틀. 로그인 상태를 모든 화면에 나눠 주고 화면을 쌓아 보여 줌 |
| `src/app/index.tsx` | 대화 화면. 캐릭터가 없으면 캐릭터 화면으로 보냄 |
| `src/app/character.tsx` | 캐릭터 화면 |
| `src/lib/api.ts` | 백엔드 주소, API 호출, 답 스트리밍 읽기 |
| `src/lib/session.tsx`, `src/lib/supabase.ts` | 앱이 켜질 때 설정 읽기와 익명 로그인 |
| `src/components/` | 여러 화면이 같이 쓰는 버튼, 불러오는 중과 오류 화면 |

`src/app/` 은 Expo Router의 규칙을 따릅니다. 이 폴더의 파일 하나가 화면 하나이고, 파일 이름이 화면 주소가 됩니다. 예를 들어 `src/app/memories.tsx` 를 만들면 `/memories` 화면이 생깁니다. 화면 파일에서는 `fetch` 를 직접 부르지 않고 `src/lib/api.ts` 를 거치게 한 것은, 백엔드 주소와 로그인 토큰을 붙이는 일을 한곳에 모으기 위해서입니다.

# 앱이 켜질 때부터 답을 받을 때까지

아래 시퀀스 다이어그램은 앱을 처음 켜서 메시지 하나의 답을 받을 때까지 앱, 백엔드, Supabase 사이에 오가는 요청을 시간 순서대로 그린 것입니다. 세로 점선이 각 참여자의 시간 축이고, 실선 화살표가 요청, 점선 화살표가 응답입니다.

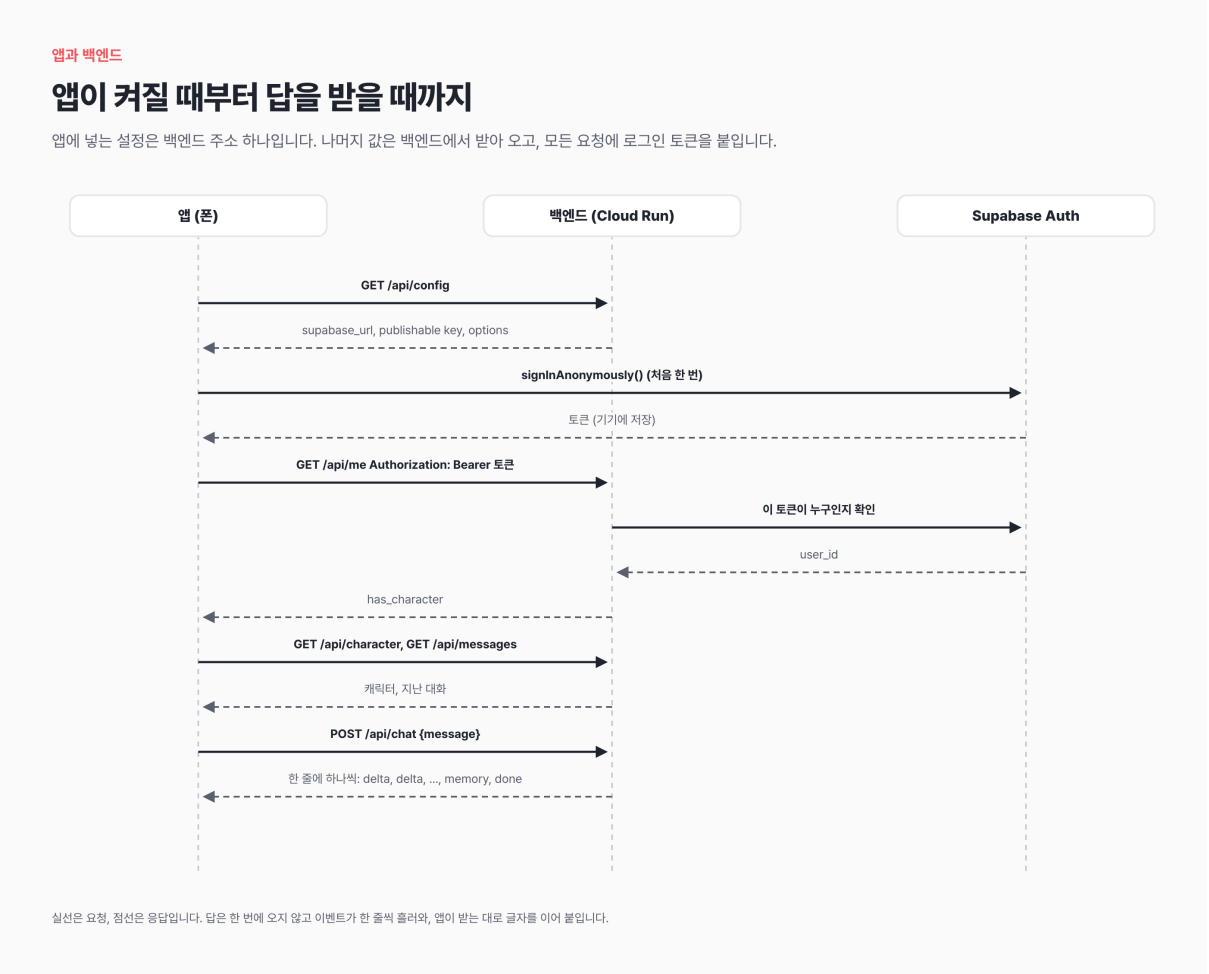



*그림: 앱이 설정을 받고 익명 로그인한 뒤 토큰을 붙여 내 정보, 캐릭터, 대화를 받고 메시지의 답을 스트리밍으로 받는 시퀀스 다이어그램*

앱에 넣는 설정은 백엔드 주소(`EXPO_PUBLIC_API_URL`) 하나뿐입니다. 앱은 먼저 `/api/config`에서 Supabase 주소와 publishable key, 캐릭터 화면에 쓸 선택지를 받아 옵니다. 백엔드가 Supabase 로그인을 쓰면 앱은 익명 로그인으로 게스트 사용자를 만들고, 받은 토큰을 기기에 저장해 앱을 껐다 켜도 같은 게스트로 이어지게 합니다. 그 뒤 모든 요청에 이 토큰을 붙이고, 백엔드는 토큰이 누구인지 Supabase에 확인해 그 사용자의 데이터만 돌려줍니다. 웹 화면도 똑같이 동작하므로, 같은 사람이 웹과 앱을 쓰면 브라우저와 폰에서 각각 다른 게스트가 됩니다.

메시지를 보내면 `/api/chat`의 답은 한 번에 오지 않고, 한 줄에 이벤트 하나(JSON)씩 흘러옵니다. 답 글자 조각인 `delta`, 도구가 약속을 만들었을 때의 `reminder`, 기억 정리 결과인 `memory`, 끝을 알리는 `done` 이 차례로 옵니다. React Native의 기본 `fetch`는 이렇게 조금씩 오는 응답을 읽지 못해서, 템플릿은 Expo의 `expo/fetch`로 응답을 줄 단위로 읽어 글자를 이어 붙입니다.

# 앱에 넣으면 안 되는 값

`EXPO_PUBLIC_` 으로 시작하는 환경변수는 빌드할 때 앱 파일 안에 그대로 들어갑니다. 앱 파일은 누구나 내려받아 열어 볼 수 있으므로, 이 값은 공개되는 값이라고 생각해야 합니다. 백엔드 주소와 Supabase publishable key 는 원래 공개되어도 되는 값이라 괜찮지만, Supabase secret key 나 LLM API 키를 앱에 넣으면 앱을 받은 누구나 그 키로 모든 사용자의 데이터를 읽거나 내 비용으로 LLM을 부를 수 있습니다. 비밀값은 지금처럼 백엔드에만 두고, 앱은 백엔드를 거쳐서만 씁니다.

**객관식 퀴즈**

### Q. 앱의 `.env` 에 `EXPO_PUBLIC_` 으로 시작하는 이름으로 넣어도 되는 값은 무엇인가요?

- 1. Supabase secret key
- 2. LLM API 키
- 3. 백엔드 주소
- 4. 백엔드가 쓰는 데이터베이스 비밀번호

- •

  1. Supabase secret key

- •

  - . LLM API 키

- •

  3. 백엔드 주소

- •

  4. 백엔드가 쓰는 데이터베이스 비밀번호

Chapter07

# 백엔드 주소 넣기

저장소 루트의 `.env.example` 을 `.env` 로 복사하고, `EXPO_PUBLIC_API_URL` 에 백엔드 주소를 넣습니다. 주소 끝에는 `/` 를 붙이지 않습니다. 넣기 전에 브라우저로 `<백엔드 주소>/health` 를 열어 `{"status":"ok"}` 가 나오는지 먼저 봅니다. 여기서 안 열리면 앱에서도 열리지 않습니다. `.env` 는 `.gitignore` 에 들어 있어 커밋되지 않습니다.

**직접 실행할 명령: 백엔드 주소 넣기**

```
# macOS, Linux
cp .env.example .env

# Windows PowerShell
Copy-Item .env.example .env

# 편집기로 .env 를 열어 한 줄을 바꾼다
# EXPO_PUBLIC_API_URL=https://friend-agent-staging-xxxxxxxxxx-uc.a.run.app
```

# 개발 서버 켜고 QR 코드 찍기

`npx expo start` 를 실행하면 개발 서버가 켜지고 터미널에 QR 코드가 나옵니다. Android 폰에서 Expo Go를 열고 Scan QR code 로 이 코드를 찍으면, Expo Go가 노트북에서 번들을 받아 앱을 실행합니다. 노트북과 폰은 같은 와이파이에 있어야 합니다. Windows에서 처음 실행하면 Node.js의 네트워크 접근을 허용할지 묻는 방화벽 창이 뜨는데, 개인 네트워크를 허용해야 폰이 노트북에 붙을 수 있습니다.

**직접 실행할 명령: 개발 서버 켜기**

```
npx expo start

# 같은 와이파이로 연결되지 않을 때 (강의실 와이파이, 방화벽)
npx expo start --tunnel

# .env 를 바꿨는데 반영되지 않을 때 (캐시 지우고 다시 시작)
npx expo start --clear
```

**직접 확인할 곳: 내 폰에서 확인**

1. Expo Go로 QR 코드를 찍으면 진행 막대가 차오르며 번들을 받은 뒤 앱이 열립니다.
2. 처음 열면 캐릭터 화면이 나옵니다. 이름과 모습, 말투를 고르고 "저장하고 대화하기"를 누릅니다.
3. 대화 화면 위에 아바타와 이름이 보이는지 봅니다. 메시지를 보내 답이 글자 단위로 이어 나오는지 봅니다.
4. Expo Go를 완전히 닫았다가 다시 열어, 같은 대화가 이어지는지 봅니다.


12:01
하루
캐릭터
안녕! 오늘은 날씨가 좋네. 너는 어때? 뭐 하고 지내? 😊
내일 날씨 알려줘
내일 서울은 맑고, 최저 3도 최고 12도라고 해! 따뜻하게 입고 나가! ☀️
다음 공휴일은 언제야?
다음 공휴일은 크리스마스인 12월 25일이야! 그때쯤 뭐 할 계획 있어? 🎄
내일 아침 8시에 물 마시라고 알려줘
알겠어! 내일 아침 8시에 '물 마시기' 약속으로 저장했어. 잘 챙겨 마셔! 💧
메시지 보내기
보내기
123
한

*그림: Android 폰의 Expo Go, 하루친구 대화 화면: 캐릭터 이름과 아바타가 위에 보이고, 보낸 메시지와 이어져 나온 답이 말풍선으로 보이는 모습*

# 결과 해석: 같은 백엔드, 다른 게스트

앱에서 저장한 캐릭터와 대화는 백엔드에 저장됩니다. 백엔드가 Supabase를 쓴다면 Supabase의 Table Editor에서 `messages` 테이블에 방금 보낸 메시지가 쌓인 것을 볼 수 있습니다. 앱을 닫았다 열어도 대화가 이어지는 것은, 앱이 익명 로그인 토큰을 기기에 저장해 두었다가 같은 게스트로 다시 로그인하기 때문입니다.

그런데 같은 백엔드를 브라우저로 열면 앱에서 나눈 대화가 보이지 않습니다. 브라우저와 폰은 로그인 정보를 따로 저장하므로 각각 다른 게스트 사용자로 로그인하고, 백엔드는 모든 조회에 그 사용자의 ID 조건을 붙이기 때문입니다. 같은 사람이 웹과 앱에서 같은 대화를 보려면 구글 로그인처럼 기기와 상관없이 같은 사용자로 알아보는 방법이 필요합니다. 첫 메시지의 답이 유독 늦게 왔다면 쉬고 있던 백엔드 인스턴스가 켜지는 콜드 스타트 때문이고, 두 번째 메시지부터는 빨라집니다.

# 연결이 안 될 때

| 보이는 것 | 원인과 확인할 것 |
|---|---|
| QR 코드를 찍어도 계속 불러오다 실패 | 노트북과 폰이 같은 와이파이인지, Windows 방화벽에서 Node.js를 허용했는지. 강의실 와이파이는 기기끼리의 연결을 막는 경우가 많아 `npx expo start --tunnel` 로 우회 |
| `--tunnel` 에서 ngrok 관련 오류 | 터널은 외부 서비스(ngrok)를 쓴다. 안내에 따라 `@expo/ngrok` 을 설치하고 다시 실행. 백신 프로그램이 막는 경우도 있음 |
| Expo Go가 "이 버전과 호환되지 않는 프로젝트"라고 함 | 프로젝트의 SDK(58)와 Expo Go가 지원하는 SDK가 다름. Play 스토어에서 Expo Go를 업데이트 |
| 앱에 "백엔드 주소가 비어 있습니다" | `.env` 의 `EXPO_PUBLIC_API_URL`. 고친 뒤 `npx expo start --clear` |
| "익명 로그인 실패" | 백엔드가 쓰는 Supabase 프로젝트에서 Allow anonymous sign-ins 가 켜져 있는지 |
| "/api/config 요청 실패" | 백엔드 주소가 맞는지, 브라우저로 `<주소>/health` 가 열리는지 |

터널은 요청이 외부 서버를 한 번 더 거쳐서 느리고, 터널 주소는 인터넷에 공개됩니다. 같은 와이파이로 연결되면 기본 방식으로 돌아가는 편이 좋습니다.

**주관식 퀴즈**

**Q. [이해 점검] 앱에서 캐릭터를 만들고 대화한 뒤, 같은 백엔드를 노트북 브라우저로 열었더니 캐릭터 만들기 화면부터 나오고 대화도 없었습니다. 데이터가 사라진 것이 아니라는 것을 어떻게 확인할 수 있는지, 왜 이렇게 보이는지 설명해 보세요.**

Chapter08

# 저장하면 바로 바뀌는 것 확인하기

개발 서버를 켜 둔 채 `src/app/index.tsx` 에서 대화가 없을 때 보이는 문구 "첫 메시지를 보내 보세요."를 원하는 문장으로 바꾸고 저장합니다. 몇 초 안에 폰 화면이 바뀝니다. 앱을 다시 빌드하거나 설치하지 않았는데 바뀌는 것은, Expo Go가 개발 서버에서 바뀐 번들을 다시 받아 화면을 새로 그리기 때문입니다.

이번에는 일부러 괄호 하나를 지우고 저장해 봅니다. 폰에 빨간 오류 화면이 뜨고, 어느 파일 몇째 줄이 잘못됐는지 나옵니다. 괄호를 되돌려 저장하면 오류 화면이 사라집니다. 코딩 에이전트가 고친 뒤 앱이 빨간 화면이 되면, 이 오류 문구를 그대로 에이전트에게 붙여 주는 것이 가장 빠른 해결 방법입니다.

# 바꿀 화면의 구조 정하기

이제 웹 화면에는 있는데 앱에는 없는 정보를 더합니다. 웹 화면은 답 아래에 "기억했어요", "약속 알림" 같은 작은 메모를 보여 주는데, 앱은 답 글자만 보여 줍니다. 코딩 에이전트는 시키는 대로 코드를 쓰지만 어떤 데이터를 어디에 보여 줄지는 대신 정해 주지 않으므로, 구조를 먼저 정합니다. 아래 세 가지 가운데 A는 모두 하고, B와 C는 사진 기능까지 마친 뒤 시간이 되면 합니다.

| 바꿀 것 | 데이터는 어디서 오나 | 바꿀 파일 | 보여 줄 모양 |
|---|---|---|---|
| A. 답 아래 기억과 약속 메모 | `/api/chat` 스트리밍의 `memory` 이벤트(`added` 목록), `reminder` 이벤트(`summary`) | `src/lib/api.ts`(이벤트 타입), `src/app/index.tsx` | 답 말풍선 아래 회색 작은 글씨로 "🧠 기억했어요: …", "⏰ …" |
| B. 말풍선에 보낸 시각 | 지난 대화는 `/api/messages` 의 `created_at`, 새 메시지는 보낸 순간의 시각 | `src/lib/api.ts`(`Message` 타입), `src/app/index.tsx` | 말풍선 아래 "오후 3:12" |

| 바꿀 것 | 데이터는 어디서 오나 | 바꿀 파일 | 보여 줄 모양 |
|---|---|---|---|
| C. 캐릭터 관심사 | 백엔드 캐릭터의 `interests` (최대 6개, 각 20자) | `src/app/character.tsx` | 입력 칸에 쉼표로 적으면 태그로 나뉨 |

세 가지 모두 백엔드는 바꾸지 않습니다. 필요한 데이터를 백엔드가 이미 보내고 있는지 먼저 확인한 것이 이 표의 두 번째 열입니다. 앱에 필요한 데이터를 백엔드가 주지 않는다면, 앱에서 우회하지 말고 백엔드에 무엇이 필요한지부터 정합니다.

# A. 답 아래 기억과 약속 메모

표의 A 줄을 평소 말로 코딩 에이전트에게 맡깁니다. 보여 줄 모양은 원하는 대로 바꿔도 됩니다.

**코딩 에이전트에게 맡길 때: 답 아래 기억과 약속 메모**

```
대화 중에 나에 대한 걸 기억하거나 약속을 저장했을 때, 답 말풍선 아래에 회색 작은 글씨로 무엇을 기억했는지 같이 보여 주는 기능 만들어 줘.
```

구현이 끝나면 폰에서 아래 순서로 확인합니다.

**직접 확인할 곳: A 확인**

1. "나 고양이 두 마리 키워"처럼 나에 대한 사실을 보냅니다. 답 아래에 기억 메모가 붙는지 봅니다.
2. "내일 아침 8시에 물 마시라고 알려 줘"를 보냅니다. 답 아래에 약속 메모가 붙는지 봅니다.
3. 앱을 닫았다 다시 열면 지난 대화에는 메모가 없습니다. 메모는 그 턴의 스트리밍 이벤트로만 오기 때문입니다.

▼ **힌트 A**

백엔드 `app/agent.py` 의 `run_turn` 이 어떤 순서로 이벤트를 보내는지 먼저 읽어 보세요. `memory` 이벤트는 답 글자( `delta` )가 다 온 뒤에 오므로, 메모를 붙일 말풍선은 "지금 받고 있는 마지막 답"입니다. 지금 대화 화면에서 `delta` 를 마지막 말풍선에 이어 붙이는 함수가 있으니, 메모도 같은 말풍선에 붙이도록 메시지 타입에 메모 목록을 하나 더하는 구조를 생각해 보세요.

# 결과 해석: 데이터가 어디서 오는지 알고 고친 화면

기억 메모는 나에 대한 사실을 말했을 때만 붙고, 인사나 질문에는 붙지 않습니다. 백엔드가 매 턴 기억을 정리하지만, 새로 기억할 것이 없으면 `added` 목록이 비어 있기 때문입니다. 지난 대화를 다시 불러왔을 때 메모가 없는 것도 같은 이유입니다. 메모는 저장된 메시지가 아니라 그 턴에 흘러온 이벤트라서, 지난 대화에도 보이게 하려면 백엔드가 메시지와 함께 메모를 저장하고 돌려줘야 합니다. 이렇게 화면에 보이는 정보마다 어디서 오는지를 알고 있으면, 무엇을 앱에서 고치고 무엇을 백엔드에 요청할지 나눌 수 있습니다.

**객관식 퀴즈**

### Q. A를 구현한 뒤 앱을 닫았다 다시 열었더니, 지난 대화의 답 아래에 있던 기억 메모가 모두 사라졌습니다. 이유로 가장 알맞은 것은 무엇인가요?

• 1. 앱을 닫을 때 백엔드가 그 기억을 지웠다
• 2. 메모는 그 턴의 `/api/chat` 스트리밍 이벤트로만 오고, `/api/messages` 가 돌려주는 지난 대화에는 들어 있지 않다
• 3. Expo Go를 다시 열면서 익명 로그인이 풀려 다른 게스트가 되었다
• 4. 구현이 잘못되어 메모가 첫 번째 말풍선에만 붙었다**제출**

- •

  1. 앱을 닫을 때 백엔드가 그 기억을 지웠다

- •

  - . 메모는 그 턴의 `/api/chat` 스트리밍 이벤트로만 오고, `/api/messages` 가 돌려주는 지난 대화에는 들어 있지 않다

- •

  3. Expo Go를 다시 열면서 익명 로그인이 풀려 다른 게스트가 되었다

- •

  4. 구현이 잘못되어 메모가 첫 번째 말풍선에만 붙었다

# 사진으로 약속 만들기

웹 화면에는 📷 버튼이 있어서, 일정표나 안내문 사진을 올리면 백엔드가 사진 속 글자를 읽고 그 글로 약속을 만들게 할 수 있습니다. 사진 속 글자를 읽어 텍스트로 바꾸는 기술을 OCR(optical character recognition)이라고 합니다. 휴대폰에서는 종이 일정표를 그 자

리에서 찍는 일이 훨씬 자연스러워서, 앱으로 만든 이유가 가장 잘 드러나는 기능입니다. 백엔드는 이미 있으므로 이번에도 앱 화면만 만듭니다.

| 백엔드에 이미 있는 것 | 내용 |
|---|---|
| `POST /api/ocr` | 사진 파일을 multipart 폼의 `photo` 필드로 받아 글자를 읽고 `{"text": "읽은 글"}` 을 돌려줌. 로그인 토큰 필요 |
| `/api/config` 의 `ocr_enabled` | 백엔드에서 OCR이 켜져 있는지(`OCR_ENABLED=true`). 꺼져 있으면 `/api/ocr` 은 404 |
| 웹 화면의 처리 | 읽은 글 앞에 "사진에서 읽은 글이야. 여기 있는 일정 중에 알려 줘야 할 게 있으면 약속으로 저장해 줘."를 붙여 입력창에 넣고, 사용자가 확인한 뒤 보냄 |

먼저 브라우저로 내 백엔드의 `/api/config` 를 열어 `ocr_enabled` 가 `true` 인지 봅니다. `false` 라면 OCR 프로그램(tesseract)이 들어간 이미지로 배포했는지와 `OCR_ENABLED` 환경변수를 확인하고, 바로 고칠 수 없으면 강사가 알려 주는 시연용 주소로 진행합니다.

# 사진을 받는 두 가지 방법

Expo에서 사진을 받는 라이브러리는 두 가지이고, 둘 다 Expo Go에 들어 있습니다(SDK 58 기준). 오늘은 둘을 함께 써서 사용자가 고르게 합니다.

| 라이브러리 | 하는 일 | 쓰는 곳 |
|---|---|---|
| `expo-image-picker` | 폰의 사진첩을 띄워 사진 한 장을 받아 옴 | 이미 찍어 둔 일정표 사진, 화면 캡처 |
| `expo-camera` | 앱 화면 안에 카메라 미리보기를 직접 그리고 촬영 | 종이 일정표를 그 자리에서 찍을 때. 촬영 화면에 "글자가 화면 안에 들어오게 찍어 주세요" 같은 안내를 넣을 수 있음 |

두 라이브러리 모두 네이티브 기능이라 사용자의 권한이 필요합니다. Expo Go에서는 권한을 묻는 창에 Expo Go의 이름과 문구가 뜨고, `app.json` 에 적은 내 앱의 권한 설명은 빌드한 앱에서만 보입니다. `expo-camera` 의 미리보기는 실제 폰에서만 보이고, 에뮬레이터나 시뮬레이터에서는 나오지 않습니다.

# 구현할 구조 정하기

| 구성 요소 | 하는 일 |
|---|---|
| 📷 버튼 (대화 화면 입력창 옆) | `ocr_enabled` 가 `true` 일 때만 보임. 누르면 "사진첩에서 고르기"와 "카메라로 찍기" 중 하나를 고름 |
| 사진첩에서 고르기 | `expo-image-picker` 로 사진 한 장을 고름 |

| 구성 요소 | 하는 일 |
| --- | --- |
| 카메라 화면 (`src/app/camera.tsx`) | `expo-camera` 미리보기, 촬영 버튼, 닫기 버튼, 찍는 법 안내. 찍은 사진을 대화 화면으로 넘김 |
| OCR 호출 (`src/lib/api.ts`) | 사진을 `photo` 필드에 담아 `/api/ocr` 로 보내고, 로그인 토큰을 붙임 |
| 읽은 글 처리 | 웹 화면과 같은 안내 문장을 붙여 입력창에 넣음. 바로 보내지 않고 사용자가 확인하고 고친 뒤 보냄 |
| 권한을 거부했을 때 | 앱이 멈추지 않고, 설정에서 권한을 켜는 방법을 안내 |

읽은 글을 바로 보내지 않는 데는 이유가 있습니다. OCR은 손글씨, 기울어진 사진, 흐린 글자를 잘못 읽기 쉽고, 잘못 읽은 날짜로 약속이 저장되면 사용자는 틀린 시각에 알림을 받습니다. 사람이 한 번 확인하는 단계를 두면 이런 오류를 보내기 전에 고칠 수 있습니다.

**코딩 에이전트에게 맡길 때: 사진으로 약속 만들기**

```
일정표를 카메라로 찍거나 사진첩에서 골라서, 사진 속 글자를 읽어 입력창에 넣어 주는 기능 만들어 줘. 글자 읽기는 백엔드의 /api/ocr 을 쓰면 돼.
```

에이전트가 만드는 방법을 물어 오면 위의 구조 표를 근거로 답합니다. 구현이 끝나면 폰에서 아래 순서로 확인합니다.

**직접 확인할 곳: 사진 기능 실행 확인**

1. 종이에 "11월 20일 오후 3시 치과"처럼 날짜와 시각이 있는 일정을 크게 적습니다.
2. 📷 버튼에서 카메라로 찍기를 고르고, 카메라 권한을 허용한 뒤 종이를 찍습니다. 입력창에 안내 문장과 읽은 글이 들어오는지 봅니다.
3. 읽은 글이 틀렸으면 입력창에서 고친 뒤 보냅니다. 답 아래에 A에서 만든 약속 메모가 붙는지 봅니다.
4. 이번에는 사진첩에서 고르기로 일정표 사진이나 화면 캡처를 골라 같은 흐름이 되는지 봅니다.
5. 폰 설정에서 Expo Go의 카메라 권한을 끈 뒤 다시 카메라로 찍기를 눌러, 앱이 멈추지 않고 정한 안내가 보이는지 봅니다. 확인한 뒤 권한을 다시 켭니다.

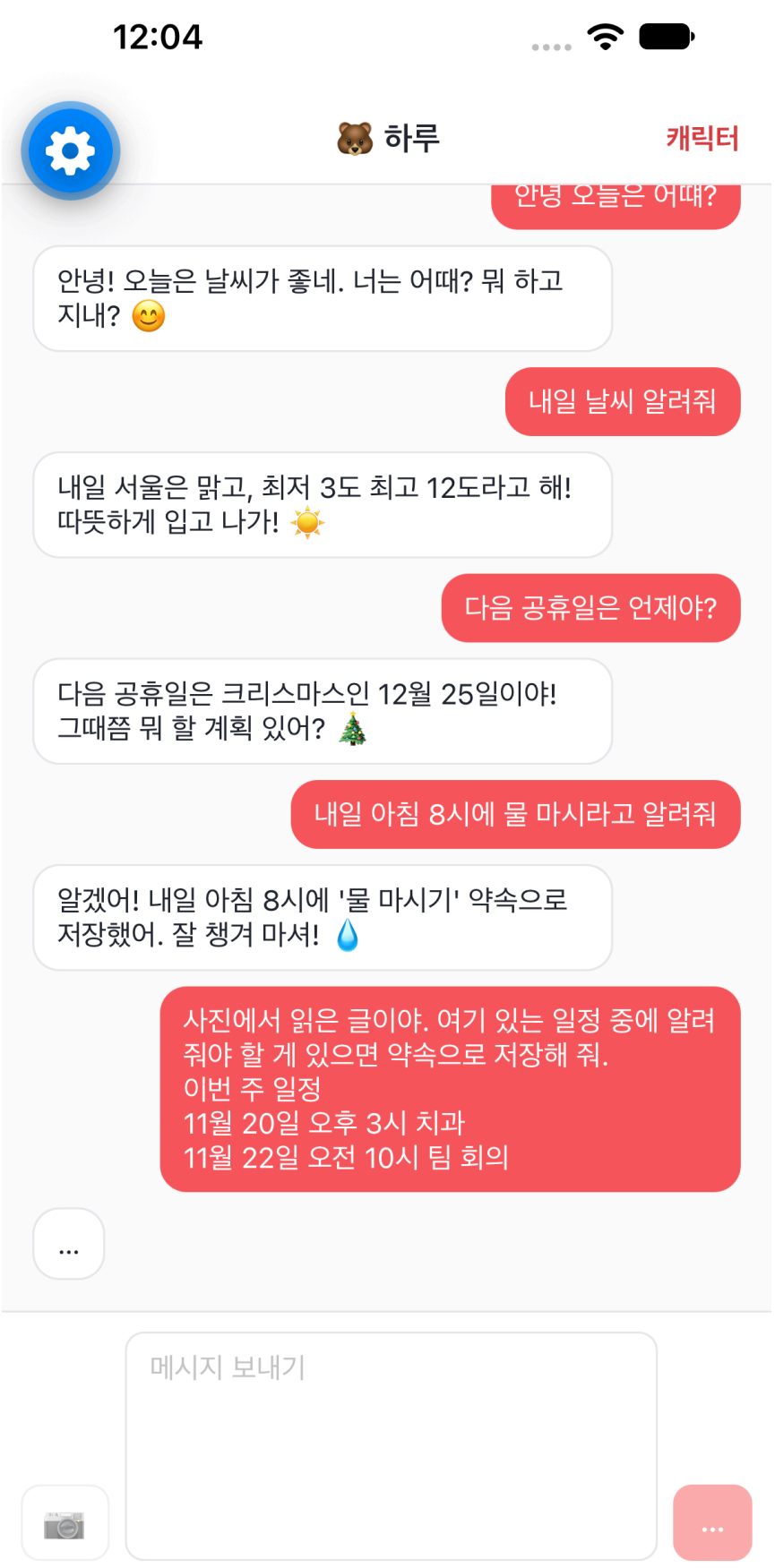

12:04
하루
캐릭터
안녕 오늘은 어때?
안녕! 오늘은 날씨가 좋네. 너는 어때? 뭐 하고 지내?
내일 날씨 알려줘
내일 서울은 맑고, 최저 3도 최고 12도라고 해! 따뜻하게 입고 나가!
다음 공휴일은 언제야?
다음 공휴일은 크리스마스인 12월 25일이야! 그때쯤 뭐 할 계획 있어?
내일 아침 8시에 물 마시라고 알려줘
알겠어! 내일 아침 8시에 '물 마시기' 약속으로 저장했어. 잘 챙겨 마셔!
사진에서 읽은 글이야. 여기 있는 일정 중에 알려줘야 할 게 있으면 약속으로 저장해 줘.
이번 주 일정
11월 20일 오후 3시 치과
11월 22일 오전 10시 팀 회의
...
메시지 보내기
...

*그림: 내 폰의 Expo Go: 카메라 화면으로 종이 일정을 찍은 뒤, 읽은 글이 대화 입력창에 들어온 모습*

# OCR이 실패할 때

사진을 보냈는데 오류가 나면 상태 코드부터 봅니다. 404이고 "사진 글자 읽기가 꺼져 있습니다"라면 백엔드에서 OCR이 꺼진 것이고, 401이면 요청에 로그인 토큰이 붙지 않은 것입니다. 400대 오류가 나며 사진이 없다는 내용이라면 사진이 `photo` 필드로 가지 않은 것으로, 템플릿의 공통 헤더에 들어 있는 JSON용 `Content-Type` 이 OCR 요청에도 붙었는지 확인합니다. multipart 폼은 경계 문자열이 들어간 `Content-Type` 을 요청이 직접 만들어야 해서, 이 헤더를 손으로 지정하면 백엔드가 사진을 찾지 못합니다. 오류 없이 엉뚱한 글이 돌아온다면 코드 문제가 아니라 사진 문제일 때가 많으므로, 밝은 곳에서 글자가 화면을 가득 채우게 다시 찍어 봅니다.

# 결과 해석: 기기 기능과 백엔드의 역할 나누기

또박또박 크게 쓴 글씨는 대체로 읽히지만, 손글씨가 흘려 쓰였거나 사진이 기울면 날짜나 시각 숫자가 틀리게 읽힙니다. 그래도 약속이 맞게 저장되는 것은 사람이 입력창에서 한 번 고쳤기 때문입니다. 이 기능에서 앱이 맡은 일은 사진을 받고 권한을 묻고 사진을 보내는 것까지이고, 글자를 읽는 일과 약속을 만드는 일은 백엔드가 합니다. 그래서 OCR 엔진을 더 좋은 것으로 바꾸거나 약속을 만드는 방식을 고쳐도 앱은 다시 빌드할 필요가 없고, 카메라 화면이나 권한 문구를 바꿀 때만 앱을 고칩니다.

**객관식 퀴즈**

### Q. 사진으로 약속 만들기를 붙인 뒤 아래 변경을 하려고 합니다. 앱 코드를 고쳐야 하는 것은 무엇인가요?

- 1. OCR 엔진을 더 정확한 것으로 바꾸기
- 2. 읽은 글로 약속을 만드는 방식 고치기
- 3. 카메라 화면의 찍는 법 안내 문구 바꾸기
- 4. 백엔드에서 OCR 끄기**제출**

- •

  1. OCR 엔진을 더 정확한 것으로 바꾸기

- •

  - . 읽은 글로 약속을 만드는 방식 고치기

- •

3. 카메라 화면의 찍는 법 안내 문구 바꾸기

- •

4. 백엔드에서 OCR 끄기

# 코딩 에이전트와 일할 때 볼 것

에이전트가 새 패키지를 설치하자고 하면, `npx expo install` 로 설치하는지(Expo SDK에 맞는 버전을 고름)와 그 패키지가 Expo Go에서 도는지 확인합니다. 네이티브 코드가 들어 있는 패키지라면 개발용 빌드가 필요해져 오늘 흐름에서 벗어납니다. 작업이 끝나면 `npx tsc --noEmit` (타입 검사)과 `npx expo-doctor` (의존성과 설정 검사)를 돌려 오류가 없는지 봅니다. 두 명령 모두 템플릿의 작업 규칙에 적혀 있어서, 에이전트도 마치기 전에 스스로 돌립니다.

Chapter09

# 오늘 배운 것

**모바일 선택지.** PWA, 웹 래핑, 네이티브, React Native와 Expo는 화면을 그리는 방식과 앱 파일을 만드는 곳이 다릅니다. Windows 노트북이 많고 백엔드가 이미 있는 조건에서는, 내 컴퓨터에 Node.js만 두고 빌드를 클라우드에 맡기는 Expo가 iOS까지 다룰 수 있는 방법이었습니다.

**Expo의 구조.** 내 코드는 JavaScript 번들로, 네이티브 부분은 Expo Go에 미리 컴파일되어 있고, 최종 앱 파일은 EAS가 클라우드에서 만듭니다. Expo Go에 없는 네이티브 기능은 개발용 빌드가 필요하고, 프로젝트와 Expo Go의 SDK 버전이 같아야 앱이 열립니다.

**앱과 백엔드.** 앱은 백엔드 주소 하나로 시작해 설정을 받고, 익명 로그인 토큰을 붙여 같은 API를 부르고, 답을 스트리밍으로 받습니다. 비밀값은 앱에 넣지 않고 백엔드에만 둡니다.

**구조를 정하고 고치기.** 화면에 보일 정보가 어디서 오는지 먼저 확인하고, 바꿀 파일과 모양을 정한 뒤 코딩 에이전트에게 맡겼습니다. 저장하면 폰 화면이 바로 바뀌고, 오류가 나면 빨간 화면의 문구로 원인을 찾았습니다.

**기기 기능 붙이기.** 사진첩과 앱 안 카메라 화면으로 받은 사진을 백엔드 OCR로 읽어 약속을 만들었습니다. 앱은 사진을 받고 권한을 묻고 보내는 일까지만 하고, 글자를 읽고 약속을 만드는 일은 백엔드에 두었으며, 잘못 읽은 글은 보내기 전에 사람이 고치게 했습니다.

**주관식 퀴즈**

### Q. [회고] 섹션 1에서 웹 서비스를 앱으로 만들 때 새로 필요한 것과 그대로 쓸 것을 나눠 적었습니다. 그 답을 다시 읽고, 오늘 실제로 새로 만든 것과 그

**대로 쓴 것을 비교해 적어 보세요.**